Proyecto SO

Proyecto final de Sistemas Operativos

Implementación de un servidor de chat multicliente en C++ sobre Raspberry Pi con autenticación, administración de usuarios y protección mediante firewall

1. Introducción

El presente proyecto consiste en el desarrollo de un sistema de comunicación local de tipo cliente-servidor, implementado en C++ sobre un sistema operativo basado en Unix, específicamente GNU/Linux, en una Raspberry Pi. El sistema permite la conexión simultánea de múltiples usuarios a un servidor de chat en una red local, e incorpora mecanismos de autenticación, control de usuarios conectados, mensajería privada, administración básica de usuarios y protección del servicio mediante firewall.

Este proyecto se desarrolló con el propósito de aplicar conceptos fundamentales de la materia de Sistemas Operativos, como manejo de procesos, concurrencia, sockets, etc.

---

2. Resumen del proyecto

Este proyecto atiende la necesidad de contar con un sistema de mensajería local que funcione dentro de una red interna, sin depender de servicios externos ni de conexión a internet. Está dirigido a estudiantes, administradores de sistemas o usuarios que deseen aprender sobre programación de redes, concurrencia, administración de usuarios y seguridad básica en entornos Unix.

La finalidad del proyecto es integrar, en una sola solución, conceptos de Sistemas Operativos y redes mediante una herramienta funcional y útil, desarrollada desde cero. Se eligió C++ por su cercanía al sistema, su compatibilidad con llamadas de bajo nivel en Linux y su buen soporte de hilos mediante `std::thread`. Asimismo, se utilizaron sockets POSIX para implementar la comunicación en red, `readline` para mejorar la interacción en terminal y `ufw` para restringir el acceso al servicio expuesto.

3. Tiempo estimado de desarrollo

Tiempo total estimado de desarrollo del proyecto: **24 horas**

Distribución aproximada:

- Planeación general del proyecto: **2 horas**
- Investigación sobre sockets y arquitectura cliente-servidor: **3 horas**
- Implementación del servidor básico: **3 horas**
- Implementación del cliente básico: **2 horas**
- Soporte multicliente con hilos: **3 horas**
- Implementación de autenticación y archivo de usuarios: **2 horas**
- Comandos del sistema (`/usuarios`, `/msg`, `/registrar`, `/expulsar`, `/salir`): **4 horas**
- Integración de autocompletado con `readline`: **2 horas**
- Configuración de firewall y pruebas de red: **2 horas**
- Pruebas, corrección de errores y ajustes finales: **1 hora**

4. Tecnologías usadas

Las tecnologías utilizadas en el desarrollo del proyecto fueron las siguientes:

- **GNU/Linux** como sistema operativo base
- **Raspberry Pi** como servidor físico
- **C++** como lenguaje principal de programación
- **Sockets POSIX** para la comunicación TCP/IP
- **Threads (`std::thread`)** para concurrencia
- **GNU Readline** para autocompletado y mejor interacción en terminal
- **UFW (Uncomplicated Firewall)** para protección del servicio
- **SSH** para administración remota del servidor
- **nmap** y **netcat** para pruebas de red y validación de puertos

5. Librerías utilizadas

Librerías estándar de C++

- `iostream`
- `thread`
- `vector`
- `string`
- `cstring`
- `algorithm`
- `mutex`
- `fstream`
- `ctime`
- `sstream`
- `unordered_map`
- `atomic`

Librerías del sistema Unix/Linux

- `unistd.h`
- `arpa/inet.h`
- `sys/socket.h`
- `netinet/in.h`

Librería adicional

- `readline/readline.h`
- `readline/history.h`

Motivo de uso de las librerías

- Las librerías estándar de C++ permitieron manejar estructuras dinámicas, archivos, hilos, cadenas de texto y sincronización entre procesos ligeros.
- Las cabeceras de sockets de Linux permitieron implementar la comunicación real entre cliente y servidor.

- `readline` se utilizó para contar con una interfaz de terminal más usable, con historial y autocompletado mediante la tecla Tab.
- `mutex` fue necesario para proteger el acceso concurrente a estructuras compartidas, como la lista de clientes conectados.
- `fstream` se usó para persistir credenciales de usuarios y guardar el historial del chat.

---

6. Lenguaje de programación utilizado

El proyecto fue desarrollado en:

- **C++**

Razón de usar C++

Se eligió C++ porque permite trabajar con estructuras y abstracciones de alto nivel, y al mismo tiempo conserva acceso directo a herramientas del sistema operativo como sockets, archivos, procesos y memoria. Es un lenguaje apropiado para comprender cómo un programa interactúa con el sistema operativo y cómo puede construir servicios concurrentes sobre Unix/Linux.

---

7. Herramientas extra utilizadas

- **G++** para compilación
- **SSH** para acceso remoto a la Raspberry Pi
- **UFW** para administración de firewall
- **nmap** para escaneo y verificación de puertos
- **netcat (`nc`)** para pruebas directas de conexión TCP
- **nano** o editor de texto en terminal para modificar archivos
- **systemd** (planeado) para desplegar el servidor como servicio del sistema
- **Raspberry Pi OS / Debian-based GNU/Linux** como entorno de ejecución

---

8. Objetivo general

Desarrollar un sistema de chat local multicliente en C++ sobre una Raspberry Pi con GNU/Linux, implementando comunicación TCP, autenticación de usuarios,

administración de sesiones concurrentes y protección del servicio mediante firewall.

---

9. Objetivos específicos

- Implementar un servidor TCP concurrente que acepte múltiples clientes.
- Permitir autenticación mediante usuario y contraseña almacenados en archivo.
- Evitar que un mismo usuario se conecte simultáneamente más de una vez.
- Incorporar comandos administrativos y de interacción para el chat.
- Implementar mensajería privada entre usuarios conectados.
- Registrar eventos del sistema en un archivo de log.
- Restringir el acceso a puertos mediante firewall.
- Mejorar la experiencia de uso del cliente con autocompletado por terminal.

---

10. Descripción funcional del sistema

El sistema está compuesto por dos programas:

10.1 Servidor

El servidor corre en la Raspberry Pi y permanece escuchando conexiones en el puerto `5000`. Cada vez que un cliente se conecta, el servidor crea un hilo independiente para atenderlo. El servidor administra la autenticación, controla la lista de clientes conectados, procesa comandos y reenvía mensajes al resto de los usuarios.

10.2 Cliente

El cliente se ejecuta desde otra máquina dentro de la misma red local. El usuario ingresa su nombre y contraseña y, si la autenticación es correcta, accede al chat. Desde ahí puede enviar mensajes públicos, mensajes privados, consultar usuarios conectados y, si tiene privilegios, registrar o expulsar usuarios.

---

11. Funcionalidades implementadas

El sistema, al momento de su entrega, cuenta con las siguientes funcionalidades:

- Conexión cliente-servidor mediante TCP
- Soporte multicliente
- Autenticación por usuario y contraseña
- Prevención de usuarios duplicados conectados
- Mensajes públicos
- Mensajes privados mediante comando `/msg`
- Consulta de usuarios conectados mediante `/usuarios`
- Registro de nuevos usuarios mediante `/registrar`
- Expulsión de usuarios mediante `/expulsar`
- Salida del chat mediante `/salir`
- Actualización dinámica de usuarios conectados
- Autocompletado de comandos y nombres de usuarios en terminal
- Bitácora de actividad en archivo `chat.log`
- Restricción de puertos con firewall

---

12. Comandos disponibles en el sistema

Comando	Función
`/usuarios`	Muestra la lista de usuarios conectados
`/msg usuario mensajet`	Envía un mensaje privado
`/registrar usuario contraseña`	Registra un nuevo usuario, solo admin
`/expulsar usuario`	Expulsa a un usuario conectado, solo admin
`/salir`	Desconecta al usuario del chat

---

13. Usuarios y permisos

El sistema de chat maneja dos niveles básicos de acceso:

Usuario normal

Puede:

- autenticarse
- enviar mensajes públicos
- enviar mensajes privados
- consultar usuarios conectados
- salir del sistema

No puede:

- registrar nuevos usuarios
- expulsar usuarios conectados

Usuario administrador

El administrador definido es:

- `faisan`

Además de las funciones normales, puede:

- registrar nuevos usuarios
- expulsar usuarios conectados

---

14. Seguridad implementada

En el proyecto se implementaron medidas básicas de seguridad:

- Autenticación obligatoria antes de ingresar al chat
- Restricción de usuarios duplicados
- Control de privilegios para acciones administrativas
- Registro de actividad en `chat.log`
- Firewall configurado para permitir solo puertos necesarios
- Control de permisos del sistema Linux para evitar que usuarios no autorizados modifiquen el servidor

Puertos permitidos

- `22/tcp` para SSH

- `5000/tcp` para el servidor de chat

Puertos bloqueados

Todos los demás puertos entrantes no autorizados.

---

15. Desarrollo del proyecto: pasos y tareas realizadas

Etapa 1. Planeación

Se definió el objetivo del sistema: construir una aplicación cliente-servidor sobre Linux que permitiera observar, de manera práctica, el comportamiento de procesos concurrentes, sockets y administración de recursos del sistema.

Etapa 2. Configuración del entorno

Se identificó la IP de la Raspberry Pi dentro de la red local y se habilitó la administración por SSH. También se verificó el compilador `g++` y las librerías necesarias.

Etapa 3. Desarrollo del servidor base

Se programó un socket TCP que escuchara conexiones en un puerto fijo. Posteriormente, se implementaron `bind()`, `listen()` y `accept()` para crear el ciclo de recepción de clientes.

Etapa 4. Desarrollo del cliente base

Se implementó un cliente que se conecta al servidor mediante IP y puerto, envía mensajes y recibe respuestas.

Etapa 5. Soporte multicliente

Se integró `std::thread` en el servidor para atender a cada cliente en un hilo independiente. Esto permitió la concurrencia de múltiples sesiones simultáneas.

Etapa 6. Sincronización y control compartido

Debido a que varios hilos acceden a la lista de clientes conectados, se incorporó `std::mutex` para evitar condiciones de carrera y corrupción de datos.

Etapa 7. Autenticación

Se creó un archivo `usuarios.txt` que almacena credenciales con el formato `usuario:contraseña`. El servidor valida dichas credenciales antes de admitir el acceso al chat.

Etapa 8. Administración de comandos

Se agregaron comandos para visualizar usuarios, enviar mensajes privados, registrar nuevos usuarios y expulsar usuarios conectados.

Etapa 9. Interfaz de terminal mejorada

Se integró la librería `readline` para permitir historial y autocompletado de comandos y usuarios.

Etapa 10. Seguridad de red

Se configuró UFW para exponer únicamente el puerto de SSH y el puerto del chat, y así minimizar la superficie de ataque del sistema.

Etapa 11. Pruebas

Se realizaron pruebas con múltiples clientes simultáneos, intentos de login erróneo, usuarios duplicados, mensajes privados, acciones administrativas y verificación del firewall mediante `nmap`.

---

16. Estructura lógica del proyecto

La estructura del proyecto es la siguiente:

```
proyecto_so/
├── servidor.cpp
├── cliente.cpp
├── usuarios.txt
├── chat.log
```

Descripción de archivos

- `servidor.cpp`: implementa la lógica del servidor, autenticación, concurrencia, comandos y administración de clientes.
- `cliente.cpp`: implementa el acceso del usuario al chat y la interacción en terminal.
- `usuarios.txt`: almacena usuarios y contraseñas.

- `chat.log`: bitácora de eventos y mensajes del sistema.

---

17. Comportamiento lógico del sistema

Flujo general de operación

1. El servidor se inicializa y comienza a escuchar conexiones en el puerto 5000.
2. Un cliente se conecta al servidor.
3. El servidor solicita usuario y contraseña.
4. Si la autenticación es válida, el cliente entra al chat.
5. El servidor crea un hilo dedicado a ese cliente.
6. El usuario puede enviar mensajes o ejecutar comandos.
7. El servidor procesa cada mensaje según corresponda.
8. Cuando el usuario sale o es expulsado, la conexión se cierra y se actualiza la lista de conectados.

---

18. Diagrama de flujo del sistema

```
Inicio
  |
  v
Iniciar servidor
  |
  v
Escuchar conexiones en puerto 5000
  |
  v
¿Se conecta un cliente?
  |---- No ----> Seguir escuchando
  |
  Sí
  |
  v
Solicitar usuario y contraseña
  |
  v
¿Credenciales válidas?
  |---- No ----> Rechazar conexión y cerrar socket
  |
  Sí
  |
```

```
  v
¿Usuario ya está conectado?
  |---- Sí ----> Rechazar conexión
  |
  No
  |
  v
Agregar cliente a lista de conectados
  |
  v
Crear hilo para atender al cliente
  |
  v
¿Recibe mensaje?
  |---- No ----> Cerrar conexión y eliminar cliente
  |
  Sí
  |
  v
¿Es comando?
  |---- No ----> Reenviar mensaje a otros usuarios
  |
  Sí
  |
  v
Procesar comando (/usuarios, /msg, /registrar, /expulsar, /salir)
  |
  v
Actualizar estado si es necesario
  |
  v
Volver a esperar mensaje
```

---

19. Diagrama UML simplificado

```
+----------------------+
|       Cliente        |
+----------------------+
| - socketCliente      |
| - nombre             |
+----------------------+
| + conectar()         |
| + autenticar()       |
| + enviarMensaje()    |
| + recibirMensajes()  |
| + ejecutarComando()  |
+----------------------+

            conecta con
                 |
                 v
```

```
+----------------------+
|      Servidor        |
+----------------------+
| - servidor_fd        |
| - clientes[]         |
| - credenciales       |
| - mutex              |
+----------------------+
| + iniciar()          |
| + aceptarClientes()  |
| + manejarCliente()   |
| + autenticarUsuario()|
| + enviarATodos()     |
| + enviarPrivado()    |
| + registrarUsuario() |
| + expulsarUsuario()  |
| + guardarLog()       |
+----------------------+

                |
                v

+----------------------+
|       Cliente        |
|   (estructura)       |
+----------------------+
| - socket             |
| - nombre             |
| - ip                 |
| - autenticado        |
+----------------------+
```

---

20. Justificación técnica

Este proyecto fue seleccionado porque integra directamente varios temas de Sistemas Operativos en una aplicación funcional. El modelo cliente-servidor permite estudiar el uso de sockets, mientras que la atención concurrente de múltiples usuarios evidencia el uso de hilos y sincronización. La autenticación y los permisos en el sistema complementan la parte de administración y seguridad.

Además, la implementación sobre una Raspberry Pi con GNU/Linux permitió trabajar en un entorno Unix real, cercano a contextos de administración de sistemas y despliegue de servicios.

---

21. Pruebas realizadas

Se realizaron las siguientes pruebas:

- Escaneo de red para localizar la IP de la Raspberry
- Verificación del puerto 5000 con `nmap`
- Prueba de conexión con `netcat`
- Inicio de múltiples clientes concurrentes
- Intentos de acceso con credenciales incorrectas
- Validación del bloqueo de usuarios duplicados
- Envío de mensajes privados
- Registro de usuarios desde el administrador
- Expulsión de usuarios conectados
- Verificación del firewall
- Validación del autocompletado con `Tab`

---

22. Resultados esperados

Se espera que el sistema:

- mantenga comunicación estable entre múltiples clientes
- solo permita acceso a usuarios autorizados
- impida la duplicidad de sesiones del mismo usuario
- permita administración básica por parte del usuario privilegiado
- registre correctamente la actividad del sistema
- limite el acceso externo mediante reglas del firewall

---

23. Limitaciones actuales

Aunque el sistema es funcional, presenta algunas limitaciones:

- Las contraseñas están almacenadas en texto plano
- El sistema se limita a una red local
- La interfaz del cliente es de terminal
- No existe cifrado de tráfico

- No se ha integrado aún como servicio permanente con `systemd`

---

24. Posibles mejoras futuras

- Hash de contraseñas en lugar de texto plano
- Integración con `systemd` para arranque automático
- Cliente con interfaz gráfica
- Registro de auditoría más detallado
- Cifrado del tráfico mediante TLS
- Persistencia más estructurada en base de datos
- Sistema de roles más completo

---

25. Conclusiones

El proyecto cumplió con el objetivo de implementar un sistema de chat multicliente sobre Unix/Linux utilizando C++. Se logró integrar comunicación TCP, concurrencia, autenticación, administración básica de usuarios y mecanismos elementales de seguridad, todo ello dentro de una arquitectura cliente-servidor funcional.

El desarrollo permitió aplicar conocimientos propios de la materia de Sistemas Operativos en un contexto realista y práctico, especialmente en lo referente a sockets, procesos ligeros, sincronización, archivos, permisos de Linux y servicios de red. Además de ser una herramienta funcional, el proyecto representa una base sólida para seguir explorando temas más avanzados como servicios persistentes, seguridad reforzada y despliegue profesional en sistemas Unix.

---

26. Anexo breve de compilación y ejecución

Compilar servidor

```
g++ servidor.cpp -o servidor -pthread
```

Compilar cliente

```
g++ cliente.cpp -o cliente -pthread -lreadline
```

Si es necesario:

```
g++ cliente.cpp -o cliente -pthread -lreadline -lncurses
```

Ejecutar servidor

```
./servidor
```

Ejecutar cliente

```
./cliente
```